```
# 8 Puzzle Problem using A* Search Algorithm

import heapq

# Goal State
goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)


# Manhattan Distance Heuristic
def h(s):

    d = 0

    for i in range(9):

        if s[i] != 0:

            j = goal.index(s[i])

            d += abs(i // 3 - j // 3) + abs(i % 3 - j % 3)

    return d


# A* Search Algorithm
def astar(start):
```

```
# (f, g, state, path)
q = [(h(start), 0, start, [])]

seen = set()

while q:

    f, g, s, path = heapq.heappop(q)

    if s in seen:
        continue

    seen.add(s)

    # Check goal state
    if s == goal:
        return path + [s]

    # Find blank tile position
    z = s.index(0)

    r, c = divmod(z, 3)

    # Possible movements
    for dr, dc in [
        (1, 0),
        (-1, 0),
        (0, 1),
        (0, -1)
    ]:
```

```
            nr = r + dr
            nc = c + dc

            if 0 <= nr < 3 and 0 <= nc < 3:

                x = nr * 3 + nc

                t = list(s)

                # Swap blank tile
                t[z], t[x] = t[x], t[z]

                t = tuple(t)

                if t not in seen:

                    heapq.heappush(
                        q,
                        (g + 1 + h(t), g + 1, t, path + [s])
                    )

    return None


# Starting State
start = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)
```

```
# Find Solution
solution = astar(start)

print("Solution:")
print()

if solution:

    for s in solution:

        print(s[0:3])
        print(s[3:6])
        print(s[6:9])
        print()

else:
    print("No solution found.")



#RUN
python astar_8_puzzle.py
```